MODEL IMPLEMENTATION AND TRAINING SUMMARY

Computer Vision and AI-Based Construction Site Monitoring via Satellite Imagery

Purpose: concise notes on the implementation completed so far, including model setup, dataset preparation, code changes, training settings, results, and limitations for the viva.

1. Implementation completed

- Set up the official BIT_CD implementation of the Bitemporal Image Transformer (BIT) for image change detection.
- Set up a Python virtual environment and CUDA-enabled PyTorch on Windows 11.
- Downloaded and prepared a subset of the LEVIR-CD building-change dataset.
- Ran the repository demo using the supplied pretrained checkpoint.
- Adjusted dataset paths to load local training, validation, and test folders.
- Fine-tuned pretrained BIT weights, evaluated the model, and generated prediction visualizations.
- Opened the visualizations locally; the predicted change patterns looked promising.

2. Environment

Item	Details
Operating system	Windows 11
GPU	NVIDIA GeForce RTX 4060 Laptop GPU, 8 GB VRAM
Python	Python 3.11.9
PyTorch	2.14.1+cu126, as reported during setup
Virtual environment	E:\AI-Construction-Monitoring\.venv
Repository	E:\AI-Construction-Monitoring\BIT_CD
Dataset	E:\AI-Construction-Monitoring\LEVIR-CD

GPU check: torch.cuda.is_available() returned True and the GPU name was NVIDIA GeForce RTX 4060 Laptop GPU. During training, nvidia-smi showed about 1,449 MiB GPU memory use, 45% utilization, 53°C, and a Python compute process. These are readings from one moment, not averages.

3. Model and dataset

BIT model

- BIT stands for Bitemporal Image Transformer. It compares two images of the same area from different times and predicts a pixel-level change mask.
- Used the official BIT_CD repository and the pretrained LEVIR checkpoint.
- Selected architecture: base_transformer_pos_s4_dd8_dedim8, matching the supplied checkpoint.
- Two output classes: class 0 = unchanged/background; class 1 = changed.
- During diagnosis, output shape was (1, 2, 256, 256): one pair, two classes, and a 256 × 256 prediction.

LEVIR-CD subset

- Subset location: E:\AI-Construction-Monitoring\LEVIR-CD.
- Each split has A, B, and label folders plus a list file.
- A is the earlier image; B is the later image; label is the ground-truth change mask.
- Counts: 445 training pairs, 64 validation pairs, and 128 test pairs (637 total). This is a subset, not the full original LEVIR-CD dataset.
- Original images are 1024 × 1024; inputs were resized to 256 × 256 for this run.
- RGB normalization used mean [0.5, 0.5, 0.5] and standard deviation [0.5, 0.5, 0.5].
- Label masks use 0 for unchanged and 255 for changed; labels are normalized by dividing by 255.

4. Code and repository adjustments

- In data_config.py, set the LEVIR root_dir to the local LEVIR-CD folder.
- In utils.py, adjusted get_loaders() so training reads LEVIR-CD\train and validation reads LEVIR-CD\val. Evaluation uses the test split.
- In datasets/CD_dataset.py, changed np.loadtxt(dataset_path, dtype=str) to np.loadtxt(dataset_path, dtype=str, ndmin=1) to handle a one-item image list correctly.
- In models/evaluator.py, added weights_only=False to torch.load for compatibility with newer PyTorch checkpoint-loading behavior.
- In models/trainer.py, changed _load_checkpoint(): resume from the project's last_ckpt.pt if it exists; otherwise load checkpoints\BIT_LEVIR\best_ckpt.pt and start fine-tuning.
- Kept fine-tuned checkpoints in checkpoints\BIT_LEVIR_FT, separate from the original pretrained checkpoint.

```
# LEVIR root configured in data_config.py
if data_name == "LEVIR":
    self.root_dir = r"E:\AI-Construction-Monitoring\LEVIR-CD"
```

5. Pretrained baseline

- python demo.py successfully ran on seven repository sample pairs. The demo change-class F1 was about 0.92756, but this tiny demo subset is not representative of test performance.
- Initial evaluation of the pretrained checkpoint on the 128-pair test subset: accuracy 94.999%, mean IoU 48.876%, mean F1 51.402%, change IoU 2.761%, change F1 5.373%, change precision 74.621%, and change recall 2.787%.
- A diagnostic on test_1.png showed very few predicted changed pixels before fine-tuning, which motivated fine-tuning.

6. Fine-tuning configuration

Training command used:

```
python main_cd.py `
  --gpu_ids 0 `
  --project_name BIT_LEVIR_FT `
  --checkpoint_root checkpoints `
  --data_name LEVIR `
  --dataset CDDataset `
  --split train `
  --split_val val `
  --net_G base_transformer_pos_s4_dd8_dedim8 `
  --n_class 2 `
  --img_size 256 `
  --batch_size 2 `
  --num_workers 2 `
  --loss ce `
  --optimizer sgd `
  --lr 0.0001 `
  --max_epochs 20 `
  --lr_policy linear `
  --lr_decay_iters 20
```

- Input size: 256 × 256; batch size: 2; data-loader workers: 2.
- Loss: cross-entropy (ce). Optimizer: stochastic gradient descent (SGD).
- Learning rate: 0.0001. Schedule: up to 20 epochs with a linear learning-rate policy.
- Pretrained weights were loaded from checkpoints\BIT_LEVIR\best_ckpt.pt. Fine-tuned outputs were saved under checkpoints\BIT_LEVIR_FT.
- Both best_ckpt.pt and last_ckpt.pt were present, each about 60 MB.

7. Evaluation results

Reported validation metrics near the end of training:

Metric	Value	Meaning
Accuracy	97.015%	Fraction of all pixels classified correctly

Metric	Value	Meaning
Mean IoU	74.771%	Average intersection-over-union across classes
Mean F1	83.698%	Average F1 across classes
Change IoU (class 1)	52.630%	Overlap for changed pixels
Change F1 (class 1)	68.964%	Precision/recall balance for change
Change precision	73.322%	Share of predicted change pixels that were true changes
Change recall	65.095%	Share of true changed pixels detected

Test evaluation command:

```
python eval_cd.py `
  --data_name LEVIR `
  --split test `
  --project_name BIT_LEVIR_FT `
  --checkpoint_name best_ckpt.pt `
  --batch_size 1 `
  --num_workers 0
```

Reported test metrics:

Metric	Original pretrained	Fine-tuned
Accuracy	94.999%	97.015%
Mean IoU	48.876%	74.772%
Mean F1	51.402%	83.699%
Change IoU (class 1)	2.761%	52.631%
Change F1 (class 1)	5.373%	68.965%
Change precision	74.621%	73.322%
Change recall	2.787%	65.097%

Interpretation: reported change-class F1 rose from about 5.37% to 68.97%, while change recall rose from about 2.79% to 65.10%. Precision decreased slightly. This suggests the fine-tuned model detected substantially more of the actual changed pixels on this subset.

8. Visual outputs

- Evaluation images were generated under vis\BIT_LEVIR_FT.
- Files included eval_1.jpg, eval_101.jpg, and multiple istrain_False_...jpg files.
- The user inspected the visualizations locally and reported that the change patterns looked good.
- Use the images to explain the earlier image, later image, ground-truth mask, and predicted mask if these panels appear in the visualization.

9. Limitations and points to state honestly

- The dataset is a subset of LEVIR-CD, not the full original dataset.
- Good LEVIR-CD results do not guarantee equal performance on the project's actual satellite source or real construction sites.
- The model-training milestone is implemented: setup, fine-tuning, test evaluation, and local visualization. These steps do not by themselves demonstrate live satellite retrieval, repeated site monitoring, or the complete dashboard/application.
- The evaluation log printed “loading last checkpoint...” despite the command specifying --checkpoint_name best_ckpt.pt. The test metrics are almost identical to validation metrics. The exact checkpoint-loading path should be verified before making a strong claim about which checkpoint was evaluated.
- The model detects pixel-level image changes; a change mask alone cannot prove that the change is new construction. Demolition, shadows, vegetation, and image differences may also be highlighted.

11. Quick definitions

Term	Simple explanation
Change detection	Comparing images from two times to locate changed pixels.
Bitemporal	Using two time points: earlier and later images.
Ground-truth mask	Reference label image marking true changed and unchanged pixels.

CONSTRUCTION SITE MONITORING — MODEL IMPLEMENTATION

Term	Simple explanation
Precision	Of the pixels predicted as changed, how many truly changed.
Recall	Of all truly changed pixels, how many the model detected.
F1 score	A combined measure of precision and recall.
IoU	Overlap of predicted and true change regions divided by their union.
Fine-tuning	Continuing training from pretrained weights on a target dataset.
Train/validation/test	Training updates the model; validation monitors/selects; test evaluates held-out examples.